



Universidade Estadual do Piauí - UESPI
Campus Alexandre Alves de Oliveira - Parnaíba/PI
Bacharelado em Ciência da Computação



Processos de Software

Átila Rabelo Lopes
Engenharia de Software

Processos de Software

- Conjunto de passos envolvendo atividades e recursos que produz um resultado¹
- Sequência de passos previsíveis que ajuda a criar a tempo um sistema de alta qualidade²
- Envolve um conjunto de técnicas e ferramentas da engenharia de software
- Importante para:
 - Garantir a **qualidade** do software
 - Aumentar a **produtividade** da equipe
 - Manter **prazos e orçamentos** estimados
 - Melhorar o **controle** e **evolução contínua** do sistema

Processos de Software

- O desenvolvimento de software não consiste apenas em escrever código. Ele passa por várias fases, que formam o ciclo de vida do software
- **Ciclo de Vida do Software** representa todas as fases pelas quais um software passa, desde a concepção até sua aposentadoria (descontinuação)
- **Por que é importante entender o ciclo de vida do software?**
 - Ajuda a **planejar melhor** o projeto
 - Permite a **gerência de riscos**
 - Facilita a **comunicação** entre equipe e cliente
 - Ajuda a escolher o **modelo de desenvolvimento mais adequado**

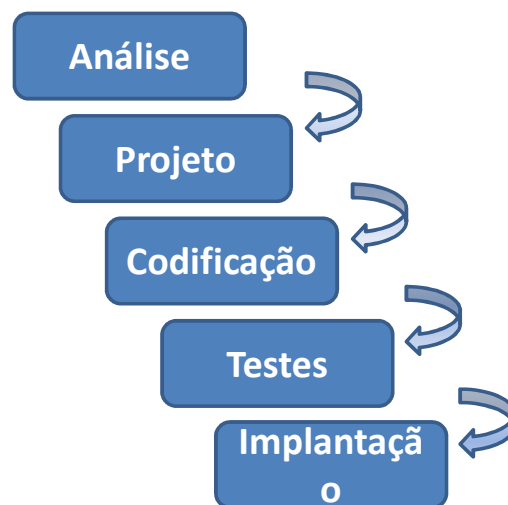
13/09/2026

3

Ciclo de Vida do Software

Etapas básicas do Ciclo de Vida de Software

- Análise (requisitos)
- Projeto (Design)
- Codificação (Implementação)
- Testes (Validação)
- Implantação



13/09/2026

4

Ciclo de Vida do Software

Análise de Requisitos (levantamento e especificação)

- O que o software deve fazer
 - Levantamento com o cliente
 - Análise de necessidades
 - Definição de funcionalidades
 - Ex: “O sistema deve permitir o cadastro de novos usuários com CPF e email únicos”

Projeto (Design)

- Como o software será construído
 - Arquitetura do sistema
 - Interface com o usuário
 - Estrutura dos dados
 - Ex: Diagramas UML, wireframes da interface

13/09/2026

5

Ciclo de Vida do Software

Codificação (Implementação)

- Desenvolvimento real do sistema
 - Programação dos módulos
 - Integração das partes
 - Uso de boas práticas (clean code, versionamento)

Testes e Validação

- Verifica se o sistema está fazendo o que deveria fazer
 - Testes unitários, de integração, de sistema e de aceitação
 - Correção de falhas

13/09/2026

6

Ciclo de Vida do Software

Implantação

- O sistema é colocado em produção
 - Treinamento dos usuários
 - Suporte inicial
 - Publicação no ambiente real

*Manutenção

- A vida após a entrega
 - Correção de bugs
 - Adição de novas funcionalidades
 - Adaptação a mudanças

13/09/2026

7

Modelos de Processo

- Modelos de Processo são formas alternativas de organizar o desenvolvimento, mais adaptáveis a contextos diferentes
- Existem diferentes tipos de modelos de processo, cada um apropriado para um determinado contexto
 - Prazo
 - Qualidade
 - Gerenciamento
 - Evolução do software
- Ajuda a equipe:
 - Planejar, gerenciar e controlar o desenvolvimento
 - Atender os requisitos
 - Cumprir prazos e orçamento

13/09/2026

8

Modelos de Processo

As atividades fundamentais dos processos (maioria dos modelos)

Atividade	Função principal
Comunicação	Levantamento e negociação dos requisitos com stakeholders
Planejamento	Estimativa de esforço, cronograma, recursos
Modelagem	Representação do software (requisitos, arquitetura, design)
Construção	Codificação + Testes
Entrega	Deploy do sistema para o cliente
Gerência de mudanças	Controle de versões, correção de erros, melhorias

13/09/2026

9

Modelos de Processo

Tipos de Processos

- Cascata (Modelo Clássico)
- Prototipação
- Incremental
- Espiral
- Desenvolvimento Ágil
 - XP (*eXtreme Programming*)
 - Scrum

13/09/2026

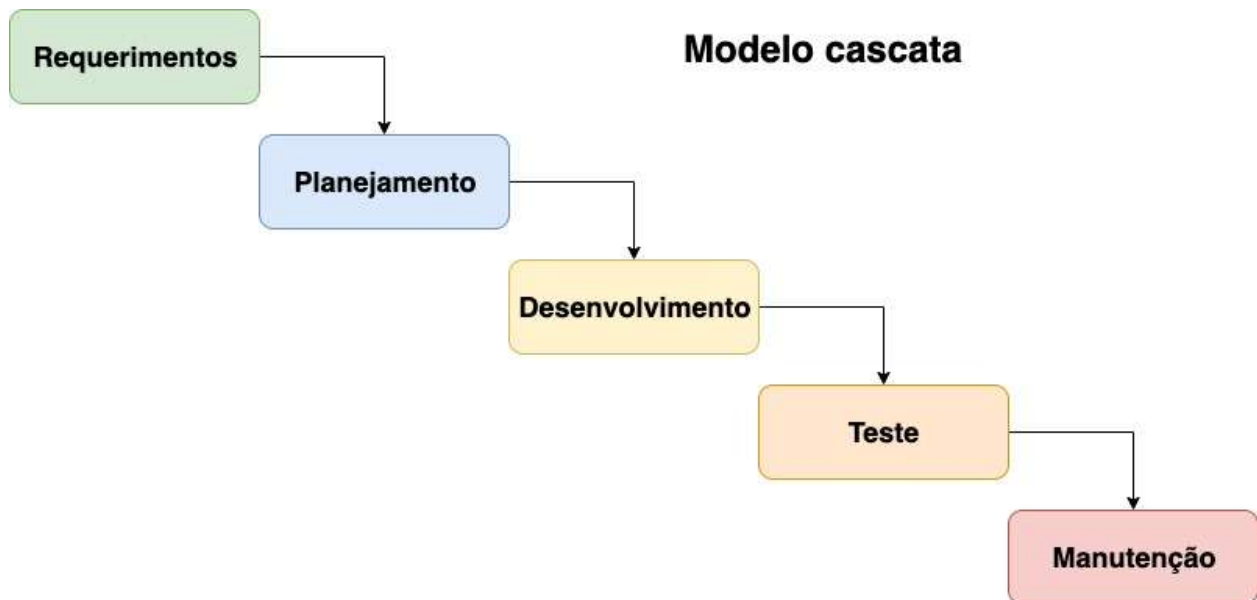
10

Modelo Cascata

Modelo Cascata

- Um dos primeiros modelos de processo
- Adota uma abordagem linear e seqüencial
- Cada fase deve ser concluída antes que a próxima comece.
 - Resultado de uma fase → Entrada da fase seguinte
- Fases:
 - Requisitos → Design → Implementação → Teste → Manutenção.

Modelo Cascata



13

Modelo Cascata - Requisitos

- Coleta dos requisitos básicos (entender as necessidades do cliente)
- Coleta dos requisitos mais intensificada
 - Compreender o domínio, funções, desempenho e interfaces exigidos
- Validação dos requisitos junto ao cliente
- Documento de requisitos

14

Modelo Cascata - Projeto

- Design/projeto
- Tradução dos requisitos para representações gráficas
- Modelagem do software
- Arquitetura do sistema

15

Modelo Cascata - Desenvolvimento

- Codificação/implementação
- Codificação das representações do projeto em uma linguagem de programação
 - Código-fonte
 - Construção do Banco de Dados / Interfaces..

16

Modelo Cascata - Testes

- Testar o comportamento do software
- Identificar erros e inconsistências
 - Teste dos aspectos lógicos do software
 - Testes das funcionalidades
- Verificar se a entrada gera os resultados esperados

17

Modelo Cascata - Manutenção

- Começa após a entrega do software ao cliente
- Etapa em que software sobre alterações/mudanças
 - Correção de erros
 - Adaptação do software ao ambiente de trabalho
 - Evolução do software (novas funcionalidades/serviços)

18

Modelo Cascata

- **Vantagem:**
 - Simplicidade e clareza no progresso
- **Desvantagem:**
 - Difícil de aplicar mudanças após o início

Por que a etapa de manutenção é mais difícil no modelo Cascata?

Como a manutenção ocorre no modelo cascata?

19

Manutenção no Modelo Cascata

- **Novo ciclo de desenvolvimento**
 - Necessário para corrigir erros ou adicionar novas funcionalidades
 - Dependendo da mudança, pode-se repetir todas as fases do modelo
- **Baixa flexibilidade**
 - Não foi projetado para acomodar mudanças frequentes ou rápidas.
 - O processo pode ficar mais lento e caro em comparação com outros
- **Documentação detalhada:**
 - Ajuda a garantir que as mudanças mantenham o padrão do software

Modelo Evolutivo

Modelo Evolutivo

- São modelos utilizados no desenvolvimento de softwares que precisam evoluir com o passar do tempo
- Consiste no desenvolvimento de forma incremental, permitindo melhorias e adaptações contínuas conforme a demanda
- Modelo iterativo
- Permite o desenvolvimento de versões cada vez mais completas

Modelo Evolutivo

Objetivo: Prover a evolução do software por meio do desenvolvimento gradativo

- Fases:
 1. Definição dos requisitos básicos.
 2. Desenvolvimento da primeira versão.
 3. Avaliação e feedback.
 4. Refinamento e adição de novas funcionalidades.
- Vantagens:
 - Facilita a adaptação a mudanças de requisitos.
 - Feedback contínuo do cliente.
- Desvantagens:
 - Pode ser difícil gerenciar um projeto em constante mudança.
 - A documentação pode ficar defasada.

23

Modelo Evolutivo

- Necessidades de evolução do software:
 - Mudanças nos requisitos do software e/ou modelo de negócio, durante o desenvolvimento
 - Prazo curto para a entrega do software completo
 - Quando os requisitos importantes já são conhecidos, mas ainda falta definir alguns detalhes

24

Modelo Evolutivo

- Tipos de Modelos Evolutivos:
 - **Modelo Incremental**
 - **Modelo Espiral**
 - **Modelo Ágil**
 - Modelo de Montagem de Componentes
 - Modelo de Desenvolvimento Concorrente
 - Processo Unificado

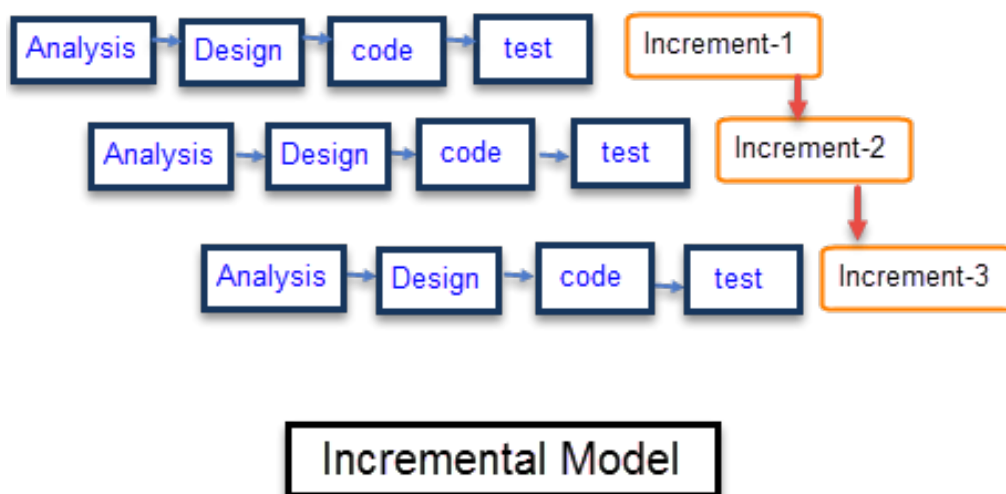
Modelo Incremental

Modelo Incremental

- O desenvolvimento é dividido em pequenas partes (Incrementos)
 - Cada parte é projetada, desenvolvida e testada de maneira independente
- Objetivo:
 - Entregar o sistema em partes, onde cada parte agrega valor ao sistema final.
- Fases:
 1. Análise/Requisitos
 2. Projeto/Design
 3. Codificação
 4. Teste

27

Modelo Incremental



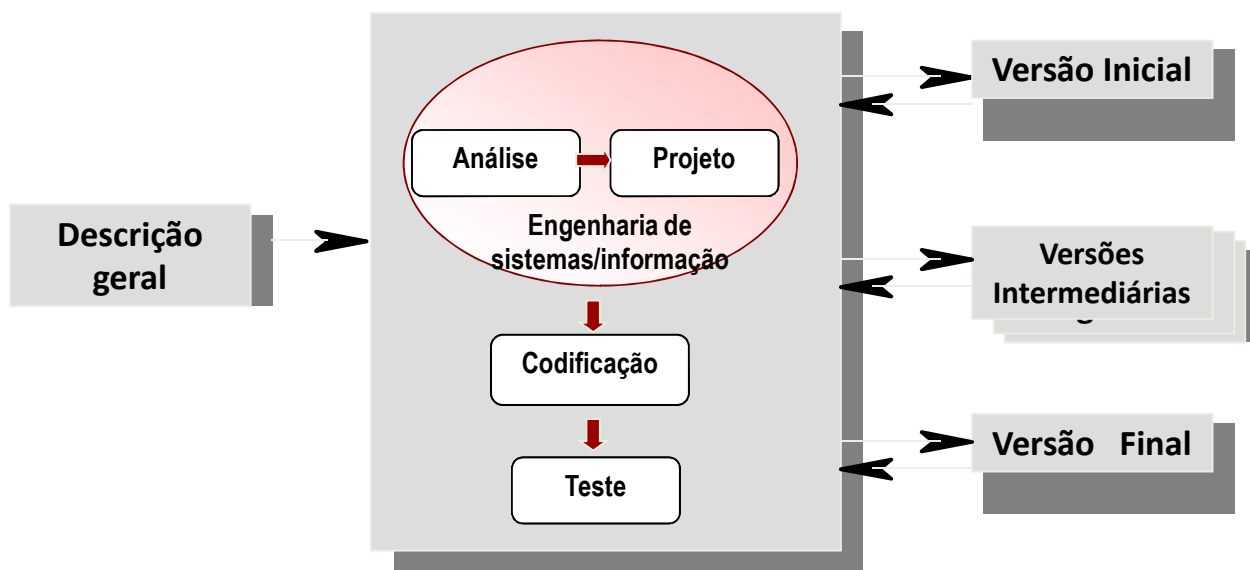
28

Modelo Incremental

- O primeiro incremento entregue é o produto principal (requisitos básicos)
- Após validado pelo cliente, novos incrementos vão sendo integrados no produto principal
- O processo se repete até a conclusão do software
- Vantagens:
 - Entregas parciais permitem testes e validações contínuas.
 - Requisitos podem ser ajustados entre incrementos.
- Desvantagens:
 - Integração contínua pode ser complexa.
 - Demora para o cliente receber o software completo
 - Requisitos não tão bem definidos inicialmente podem causar atrasos.

29

Modelo Incremental



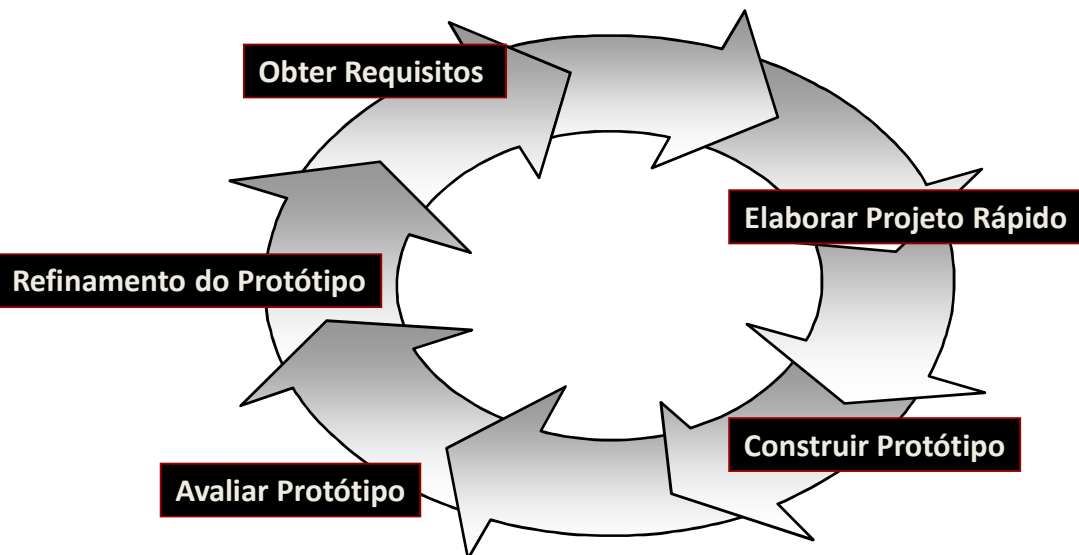
30

Modelo de Prototipação

Modelo de Prototipação

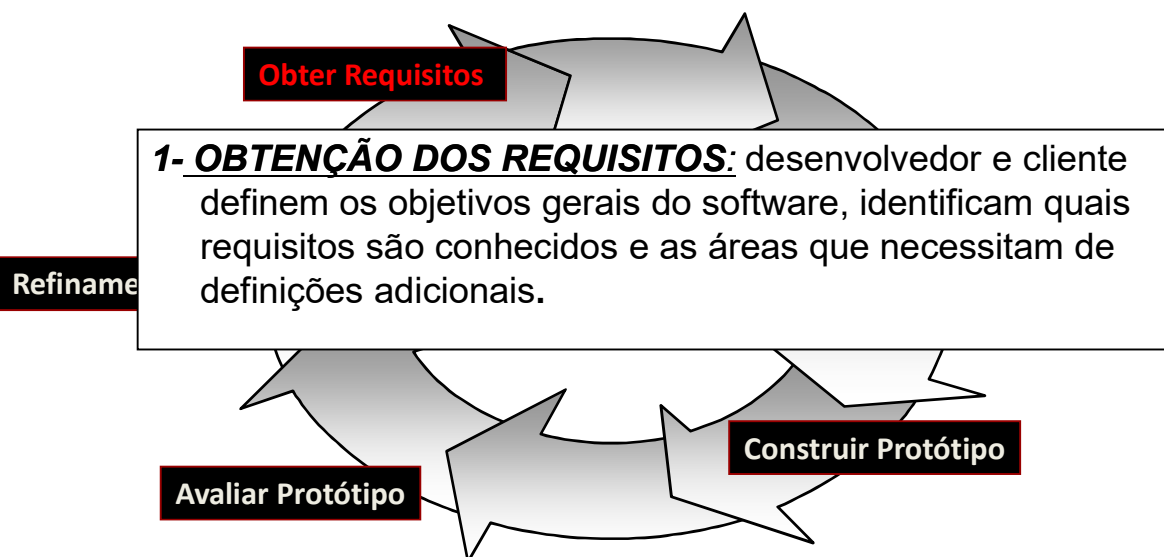
- Foco na construção rápida da versão inicial do software (protótipo) para ser avaliado pelo usuário
- Objetivo: Entender os requisitos do usuário para obter uma melhor definição dos requisitos do sistema
- Fases:
 1. Levantamento inicial de requisitos
 2. Construção do protótipo
 3. Avaliação pelo cliente
 4. Refinamento do protótipo até chegar ao sistema final

Modelo de Prototipação



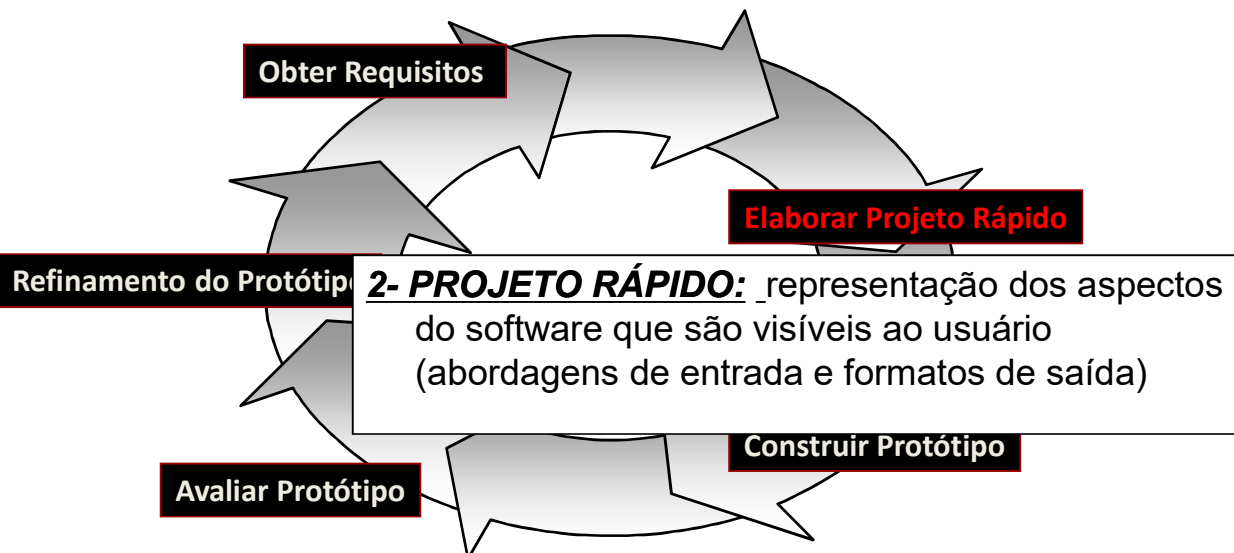
33

Modelo de Prototipação



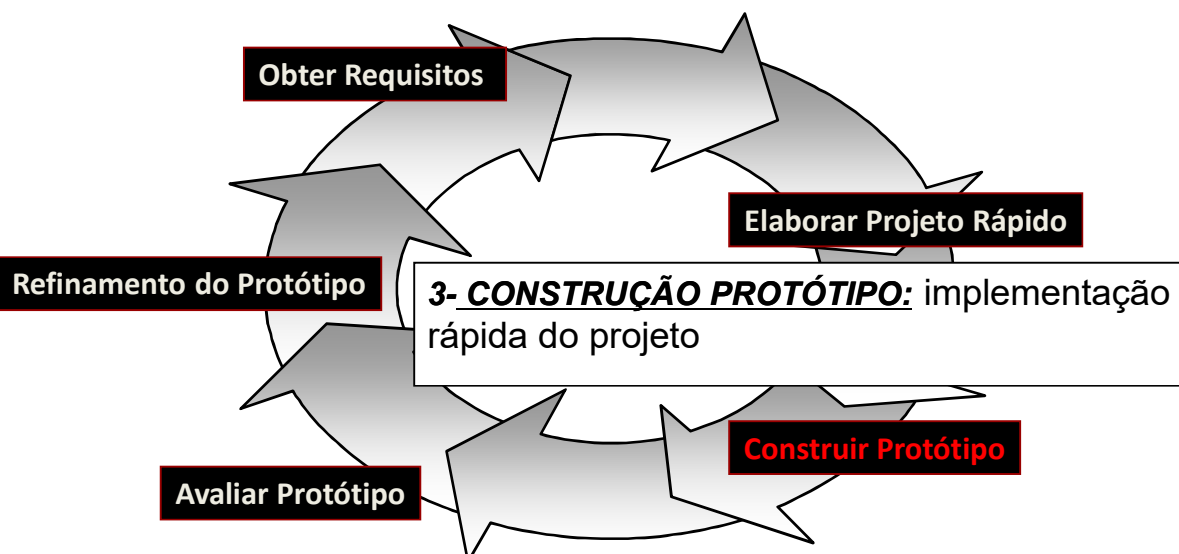
34

Modelo de Prototipação



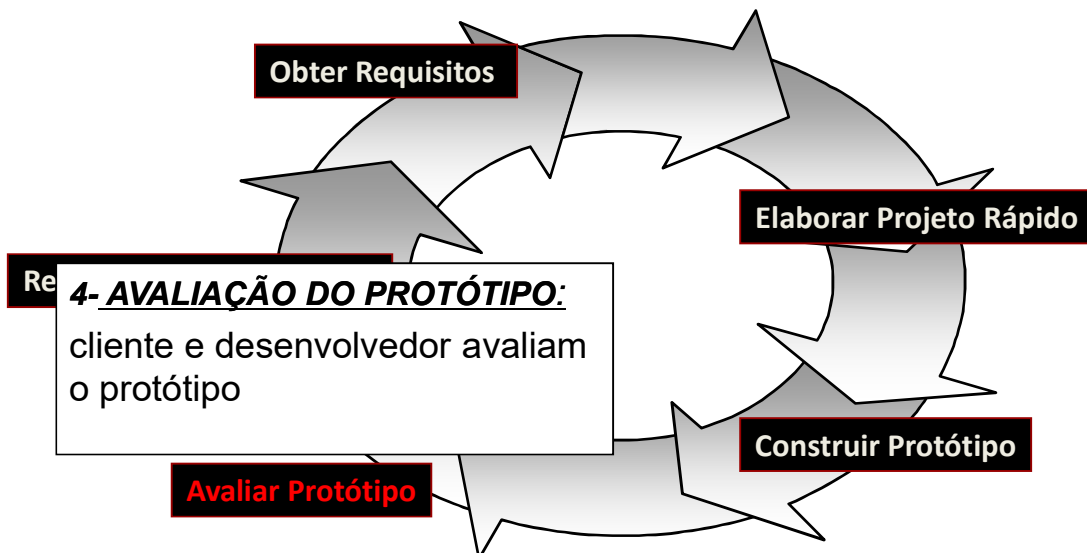
35

Modelo de Prototipação



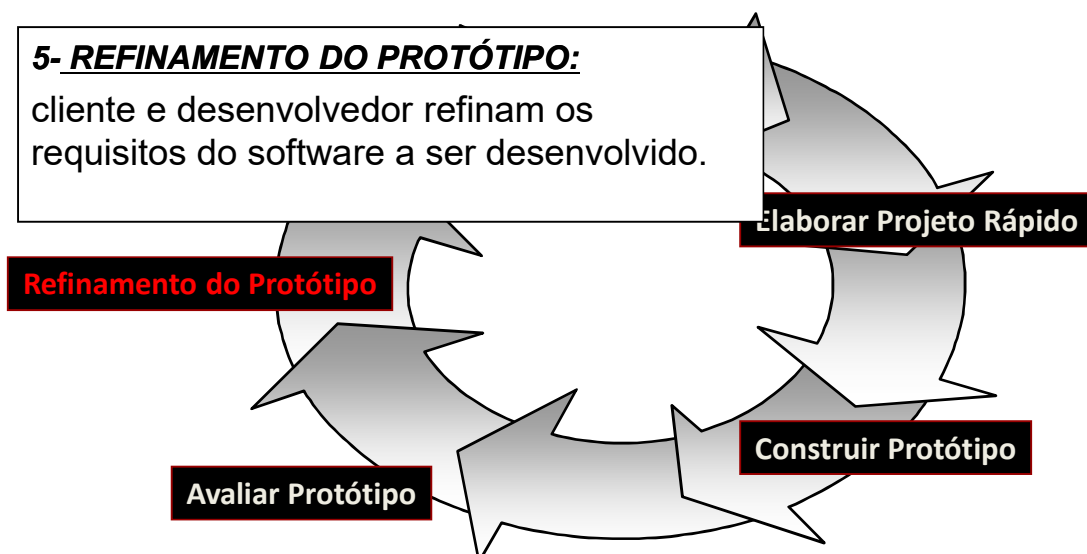
36

Modelo de Prototipação



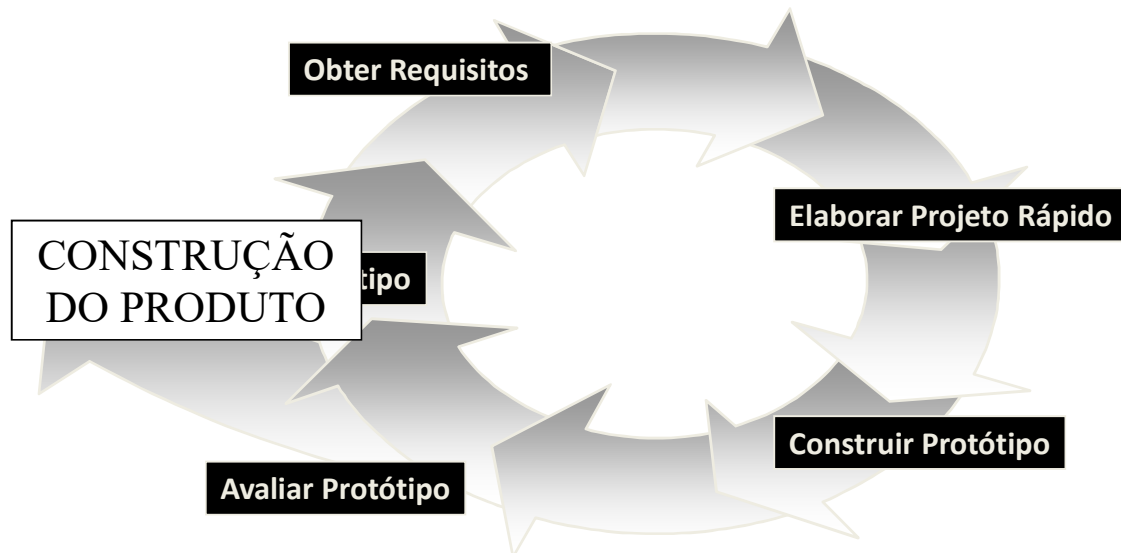
37

Modelo de Prototipação



38

Modelo de Prototipação

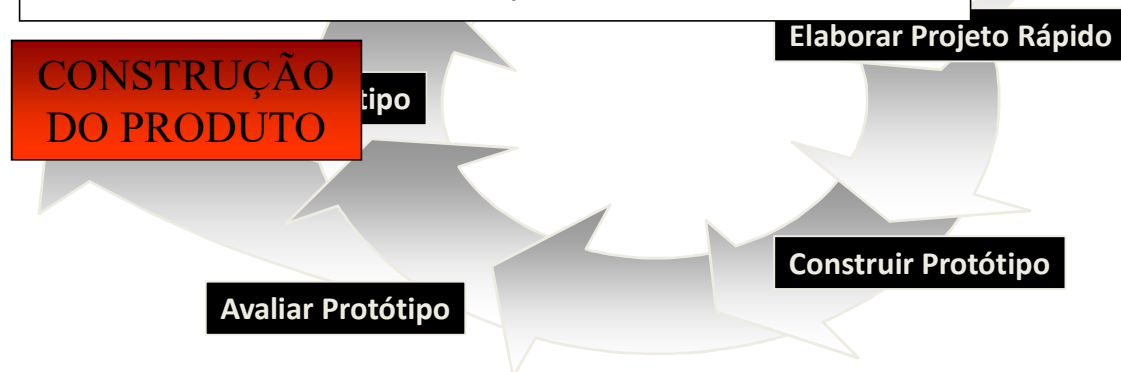


39

Modelo de Prototipação

6- CONSTRUÇÃO PRODUTO:

identificados os requisitos, o protótipo deve ser descartado e a versão de produção deve ser construída considerando os critérios de qualidade.



40

Modelo de Prototipação

- Vantagens:
 - Facilita a compreensão dos requisitos.
 - Reduz o risco da má comunicação entre desenvolvedores e clientes.
- Desvantagens:
 - Protótipos podem ser confundidos com o sistema final.
 - Se não gerenciado corretamente, pode resultar em um processo mais demorado.

41

Modelo Espiral

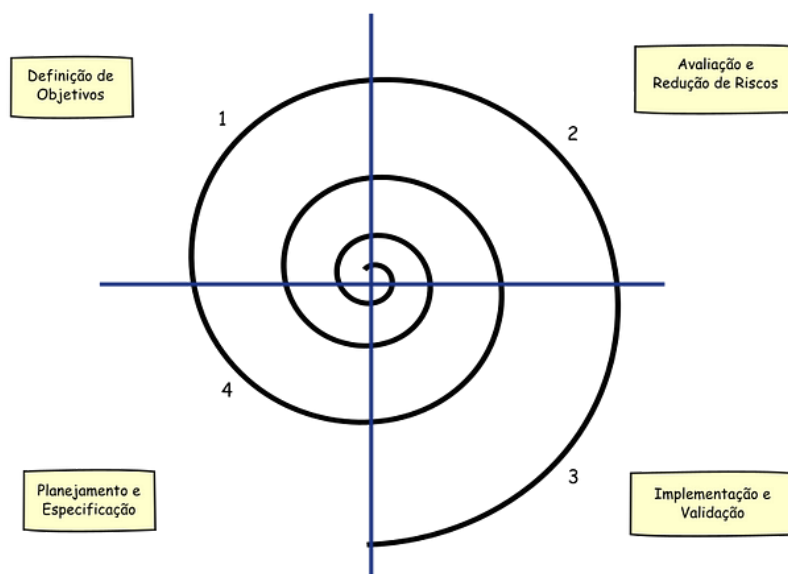
Modelo Espiral

- Incorpora aspectos do modelo incremental, cascata e prototipação, incluindo uma nova abordagem (Análise de riscos)
- O desenvolvimento ocorre em vários ciclos,
 - Cada ciclo representa uma etapa de desenvolvimento
- Os riscos vão sendo minimizados a cada nova iteração
- Usa a Prototipação em qualquer etapa da evolução do produto

Fases:

1. Definição de objetivos
2. Avaliação e análise de riscos
3. Desenvolvimento e validação (primeira iteração)
4. Planejamento da próxima iteração

Modelo Espiral



Modelo Espiral

Definição dos objetivos

- Definição dos objetivos específicos
- Identificação de restrições do projeto e produto
- Elaboração do plano de gerenciamento
- Identificação do riscos do projeto

Avaliação e redução de riscos

- Análise detalhada dos riscos identificados
- Planejamento e execução das ações para reduzir os riscos

Ex: Se houver risco de existirem requisitos inadequados, pode-se desenvolver protótipos do sistema para verificar a importância dos requisitos

Modelo Espiral

Desenvolvimento e validação

- Após a avaliação de risco, um modelo de desenvolvimento de sistema deve ser selecionado:
 - Se os riscos de interfaces forem os principais riscos identificados pode-se adotar o modelo de prototipação
 - Se o risco de integração de subsistemas for o principal risco identificado pode-se escolher o modelo em cascata

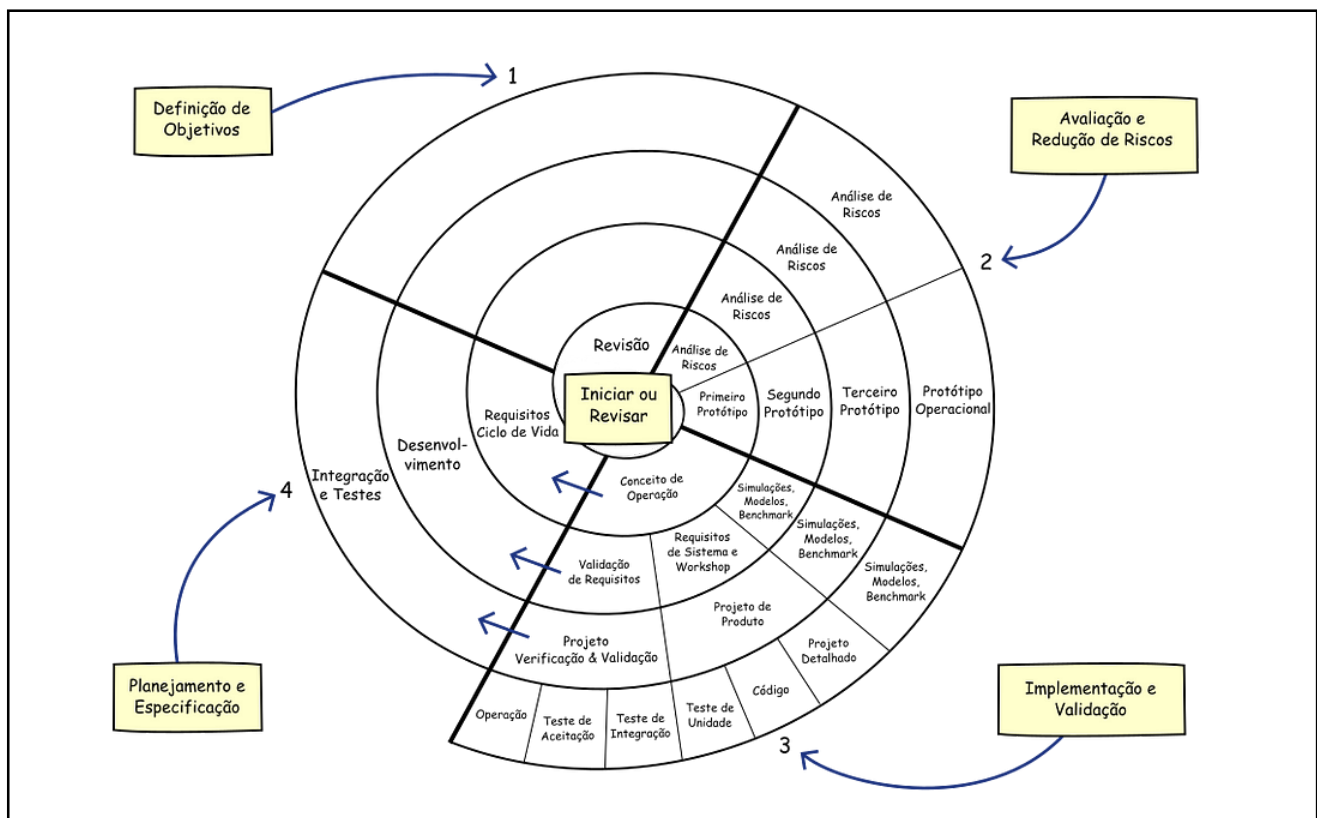
Planejamento e especificação

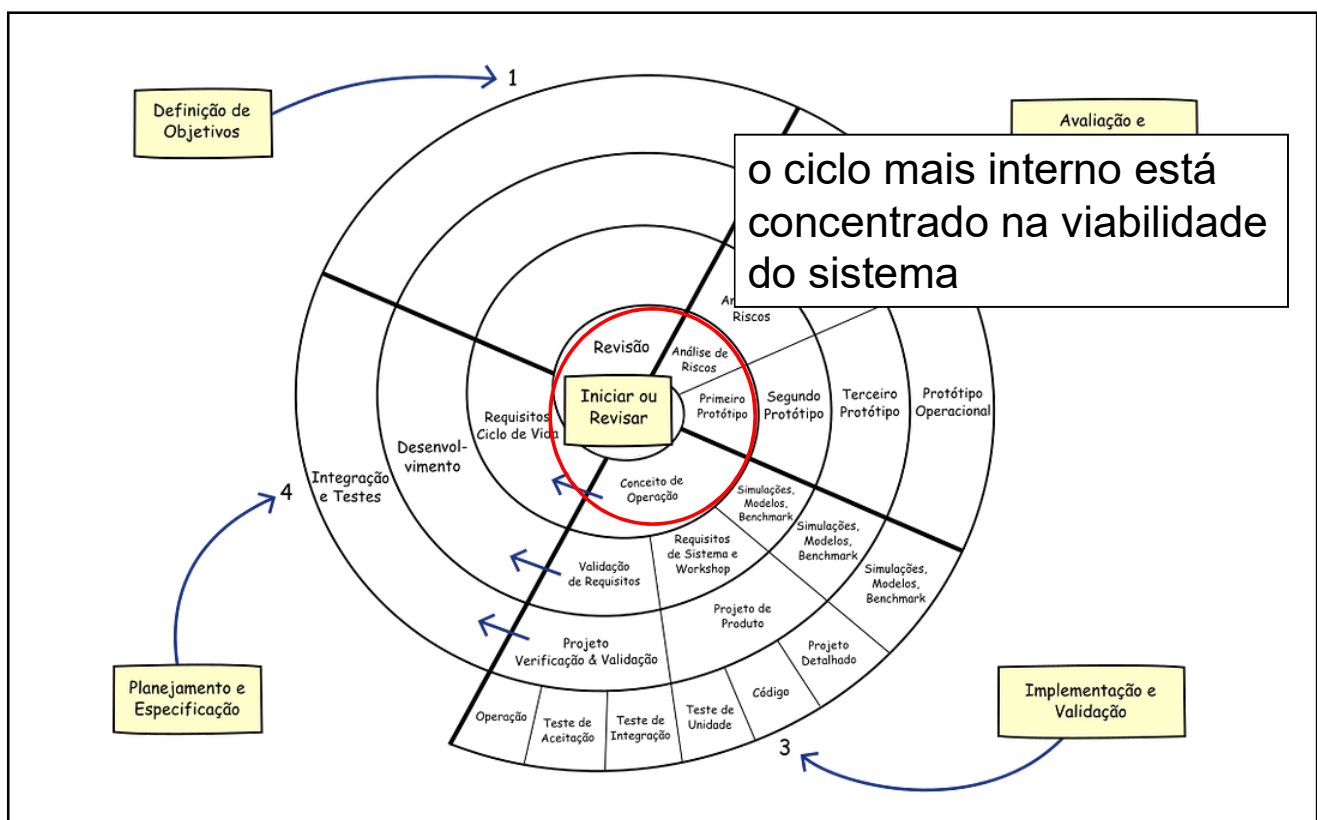
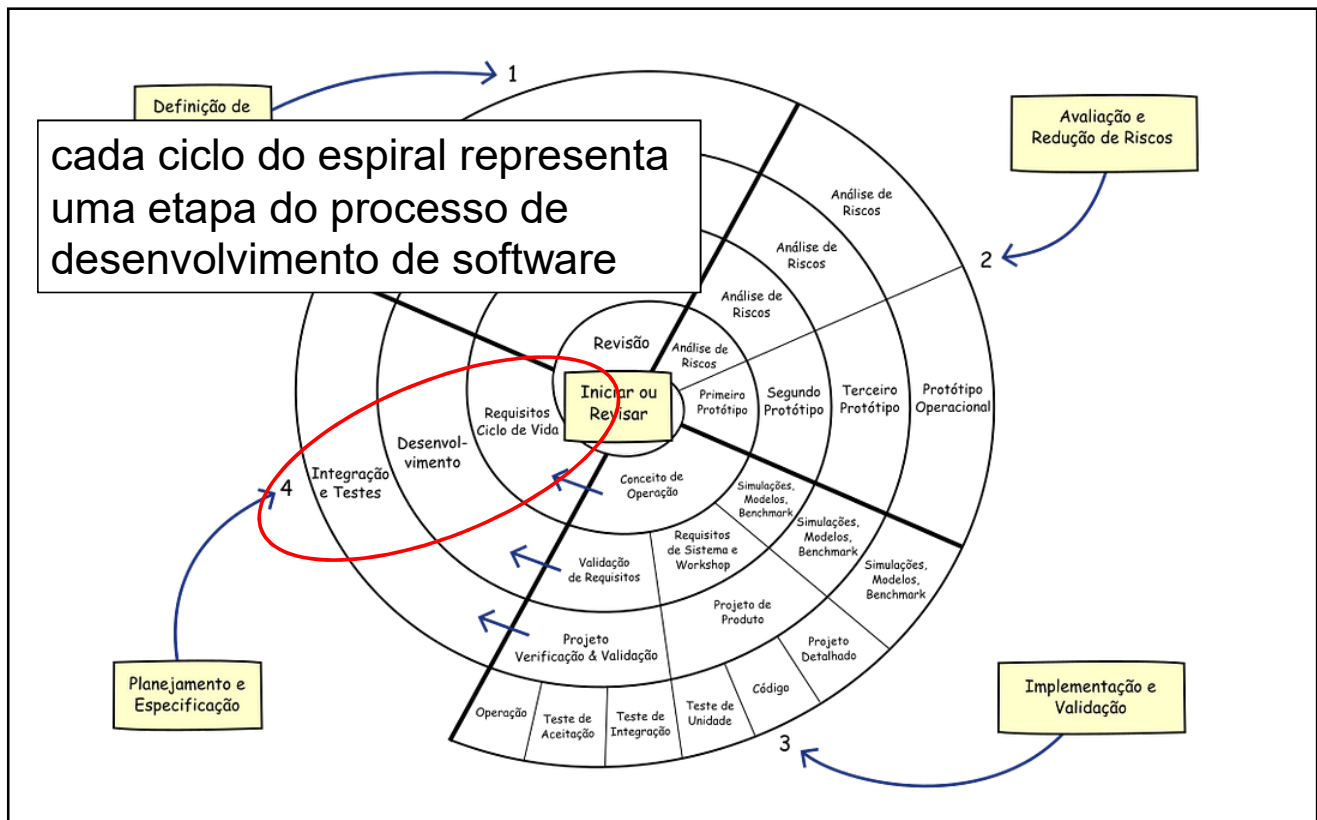
- O projeto é revisado e decidido se passa para o próximo ciclo
- Planejamento e especificação das atividades da próxima fase do projeto

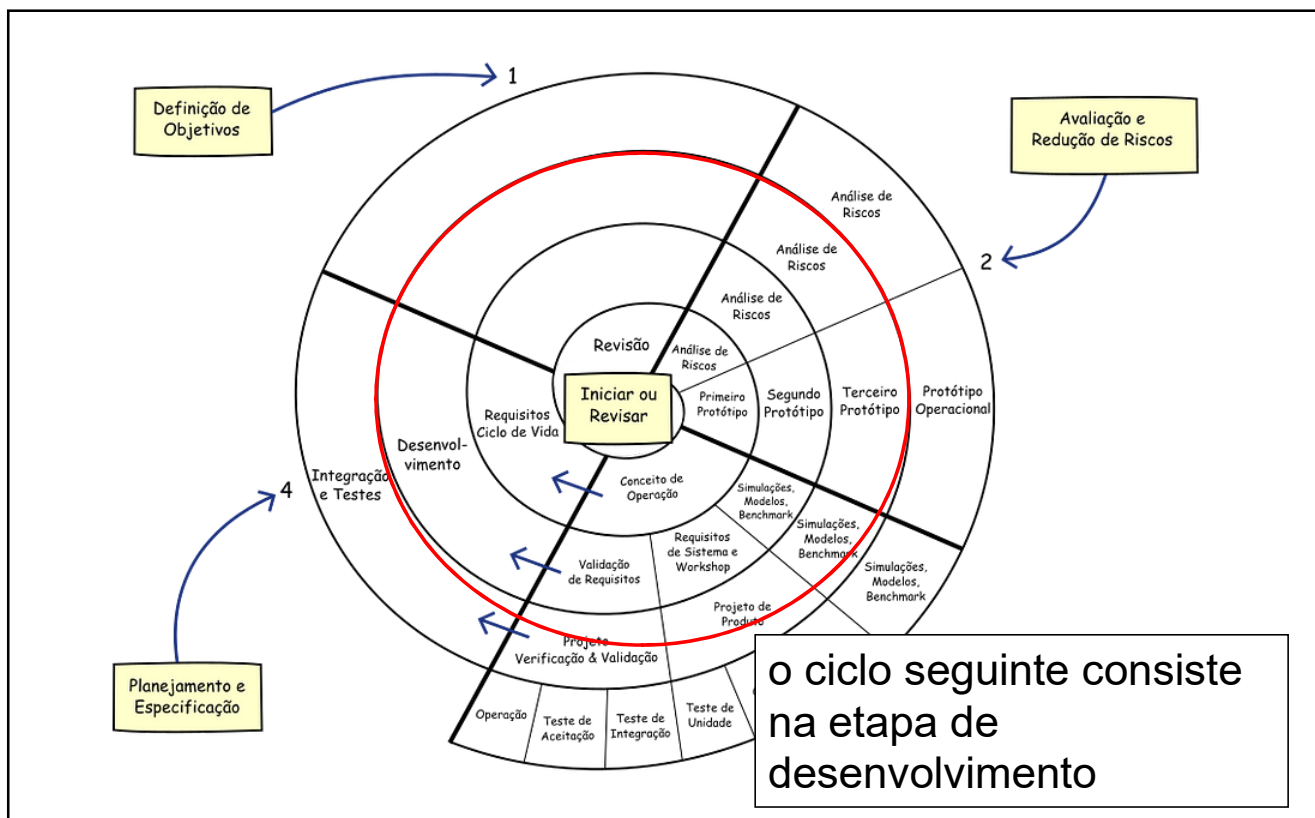
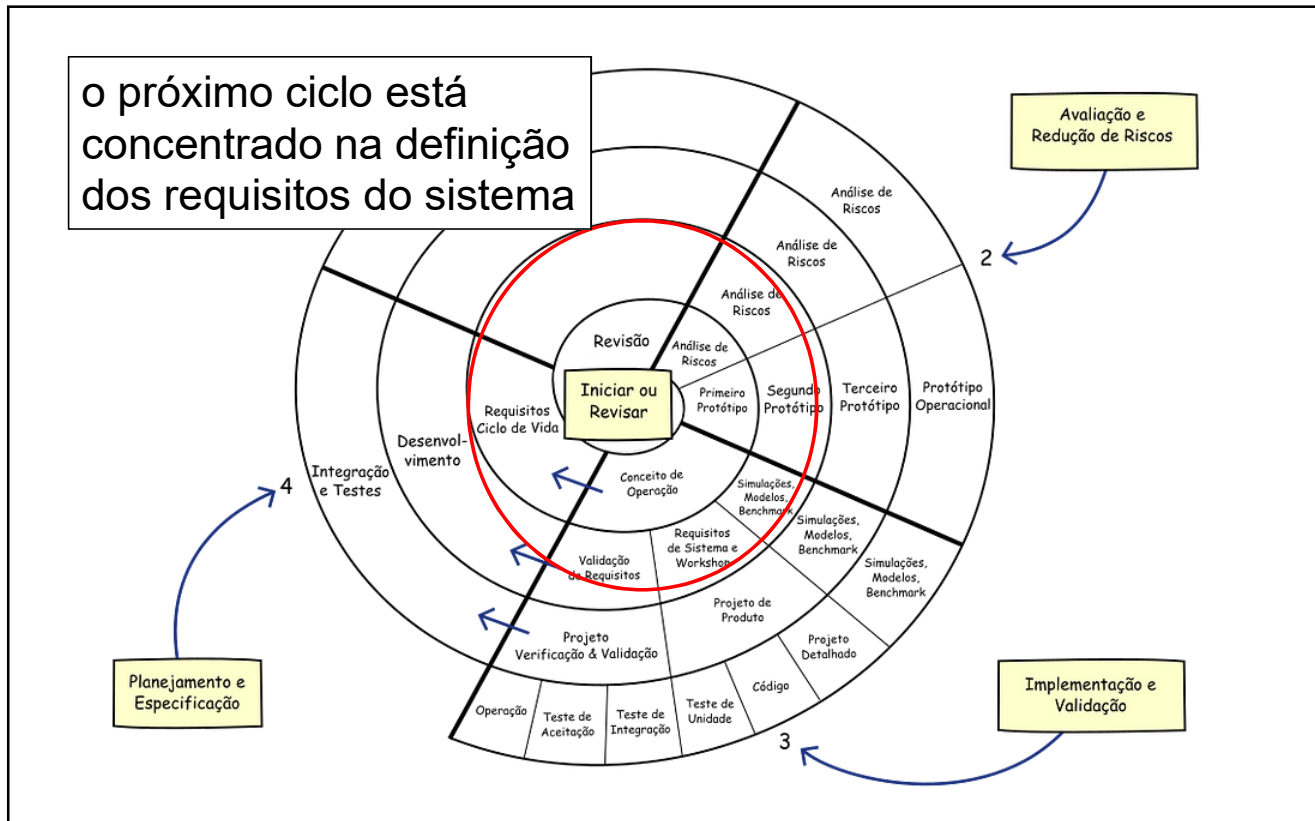
Modelo Espiral

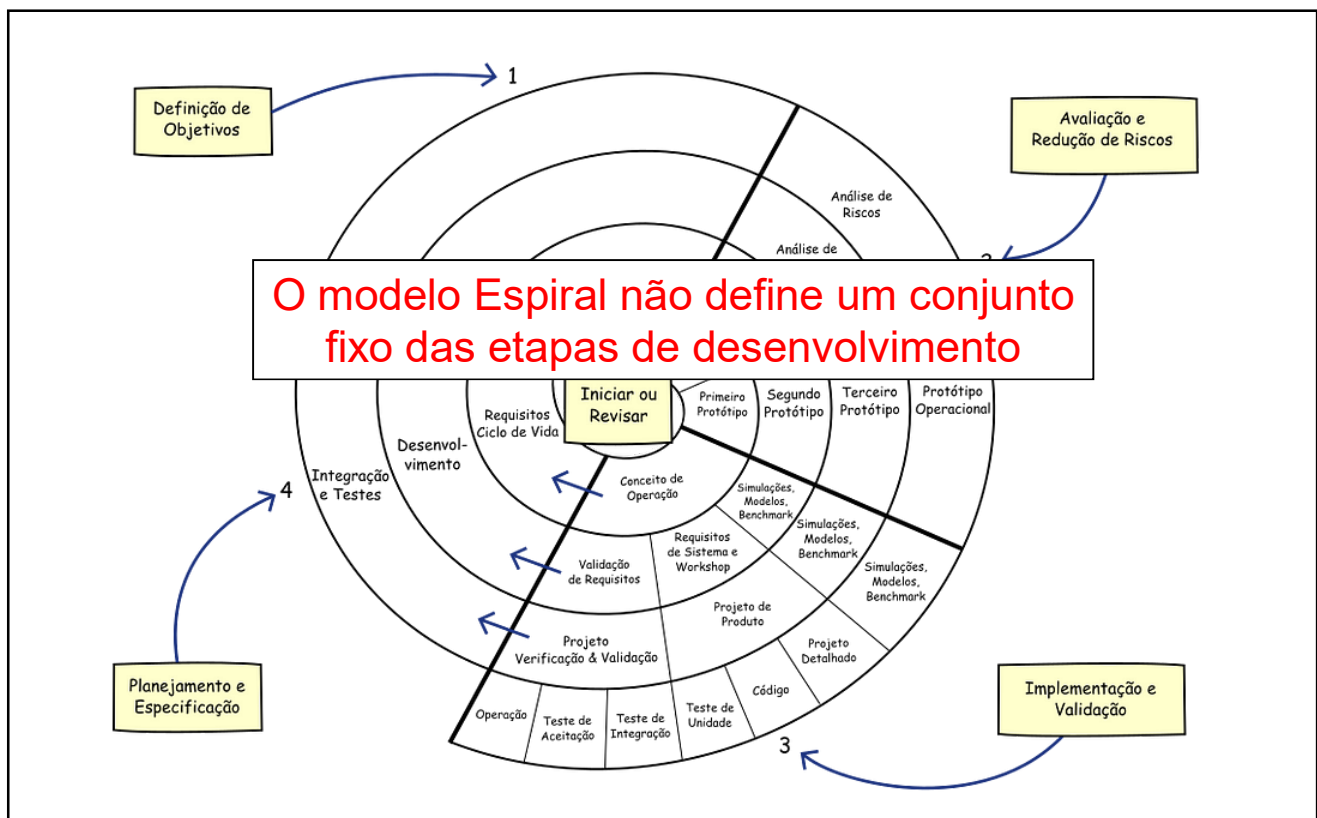
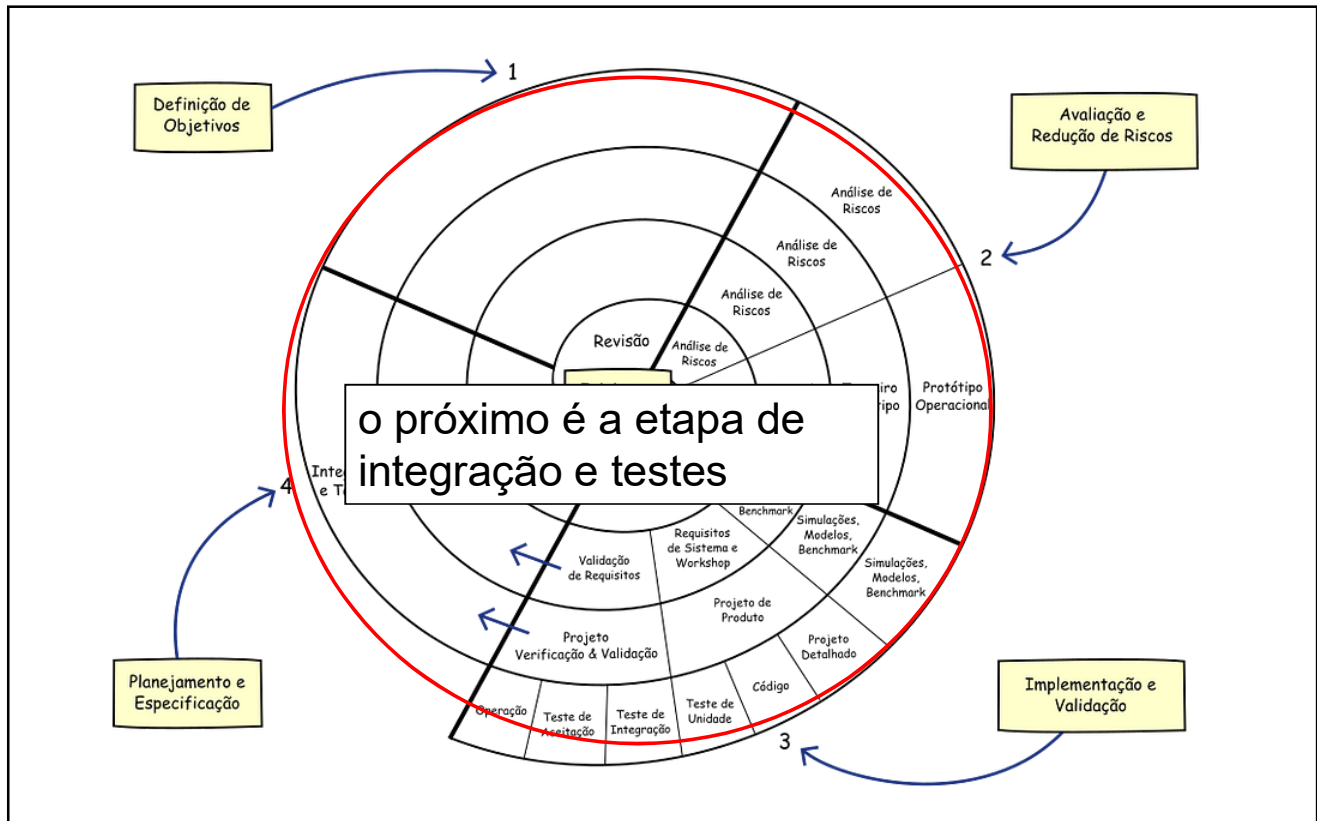
Cada ciclo do espiral envolve:

- Determinar os objetivos, alternativas e restrições relacionadas à iteração que vai se iniciar
- Identificar e resolver os riscos relacionados
- Avaliar alternativas disponíveis. Podem ser feitos protótipos para analisar a viabilidade de diferentes alternativas
- Desenvolver os artefatos relacionados à iteração corrente e valida-los
- Planejar a próxima iteração
- Obter concordância em relação ao planejamento









Modelo Espiral

Vantagens

- Adequado para projetos grandes e complexos
- Gerenciamento contínuo dos riscos
- Versátil a mudanças
- Engloba características de outros modelos

Desvantagens

- Mais difícil de gerenciar
- Exige experiência dos analistas de risco
- Pode ser mais caro e demorado (quantidade de iterações)